

Построение графиков с использованием библиотеки Matplotlib

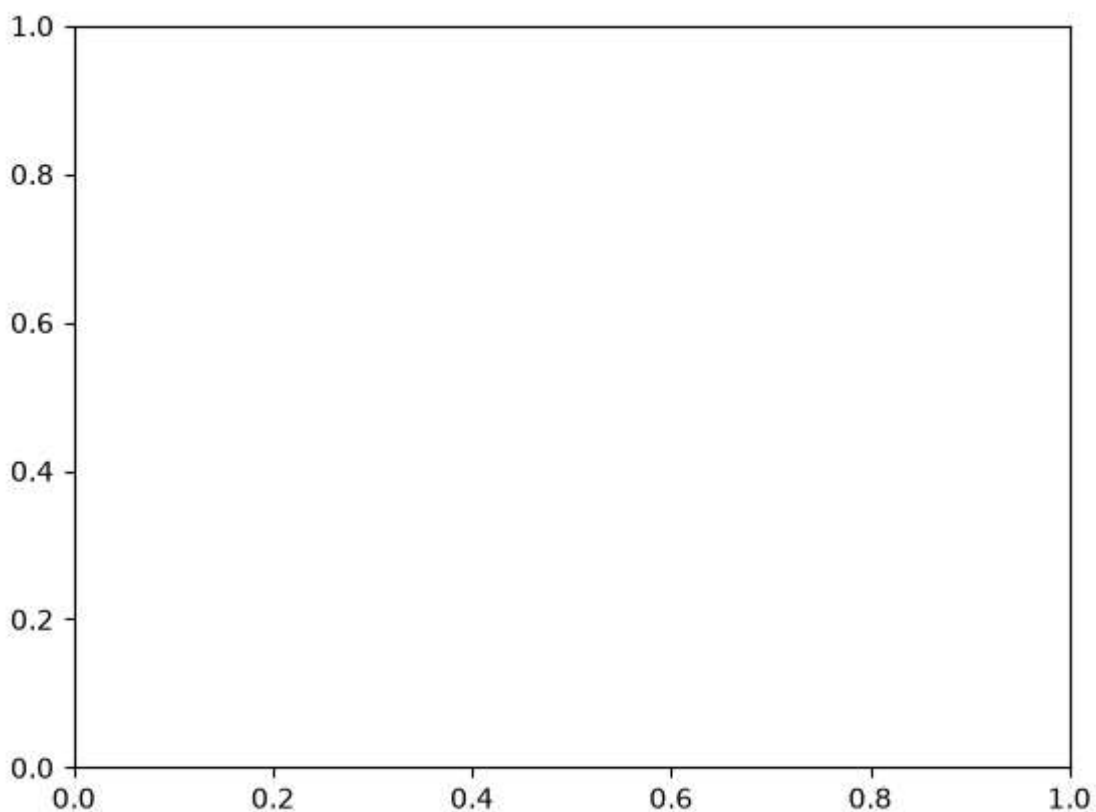
1. Начало работы

```
In [2]: import matplotlib.pyplot as plt # импорт модуля pyplot от matplotlib. В большинс
import numpy as np # импорт библиотеки numpy
%matplotlib inline # будет рисовать графики прямо в ноутбуке, под ячейкой, а не
```

В строке `fig = plt.figure()` создаётся область `Figure` (экземпляр класса `figure`). В строке `ax = fig.add_subplot(111)` к `Figure` добавляется область `Axes`. Вообще, было бы правильнее использовать `fig.add_axes`, но в данном случае `fig.add_subplot(first_plot)` намного удобнее, в конце концов `subplot` просто размещает `Axes` на сетке `Figure`. Обратите внимание на параметр, который мы передаем 'first_plot' - это первая строка, первый столбец и первая (единственная) ячейка на сетке `Figure`.

- Порядок создания: 1. Figure ---> 2. Одна или несколько Axes

```
In [3]: fig = plt.figure()
ax = fig.add_subplot(1,1,1) # 1 - количество строк, 1 - количество столбцов, 1 -
plt.show()
```



Несколько `Axes` на `Figure`

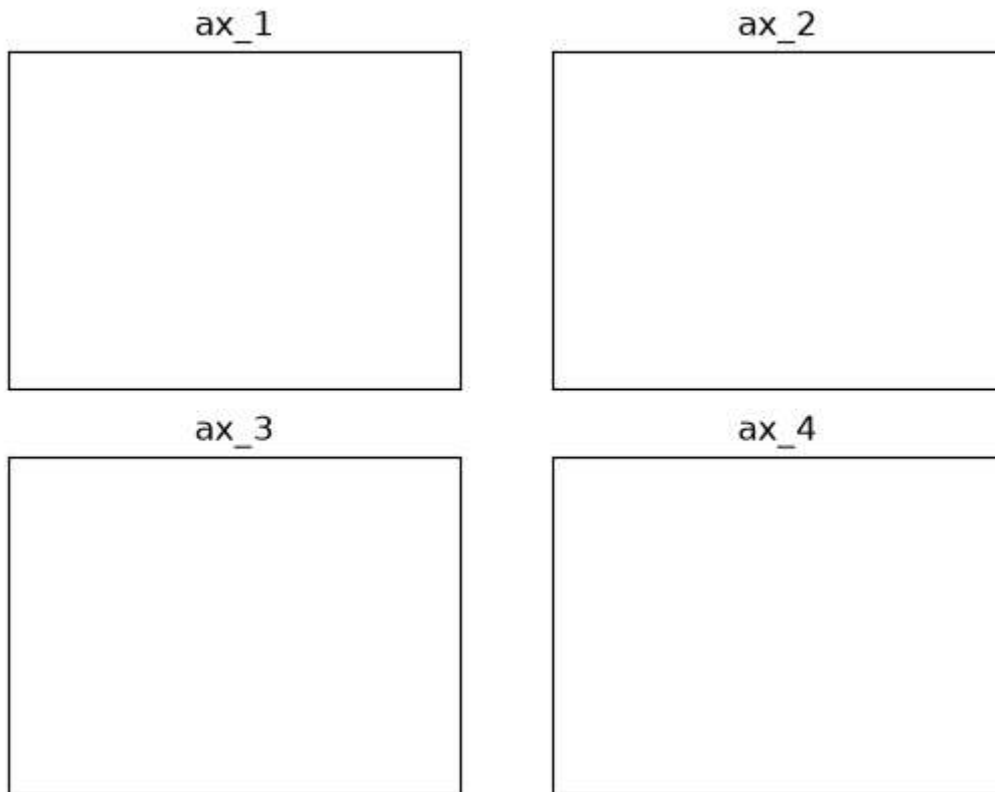
```
In [4]: import matplotlib.pyplot as plt
```

```
fig = plt.figure()

ax_1 = fig.add_subplot(2, 2, 1)
ax_2 = fig.add_subplot(2, 2, 2)
ax_3 = fig.add_subplot(2, 2, 3)
ax_4 = fig.add_subplot(2, 2, 4)

ax_1.set(title = 'ax_1', xticks=[], yticks=[])
ax_2.set(title = 'ax_2', xticks=[], yticks=[])
ax_3.set(title = 'ax_3', xticks=[], yticks=[])
ax_4.set(title = 'ax_4', xticks=[], yticks=[])

plt.show()
```



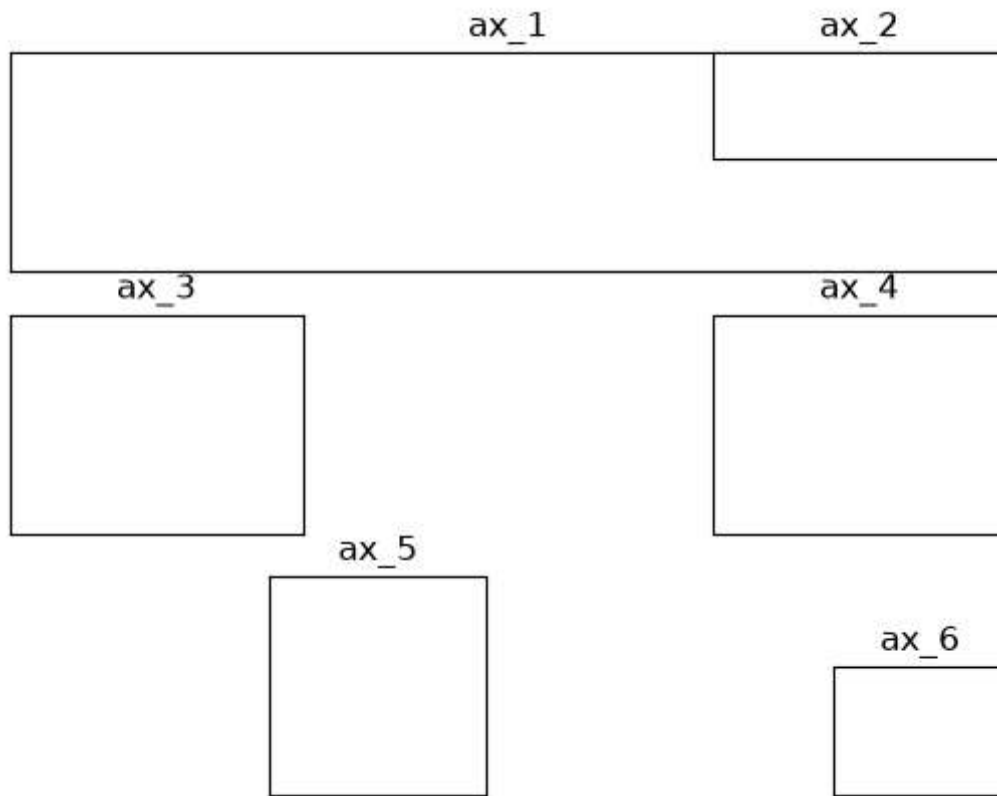
In [5]: `import matplotlib.pyplot as plt`

```
fig = plt.figure()

ax_1 = fig.add_subplot(3, 1, 1)
ax_2 = fig.add_subplot(6, 3, 3)
ax_3 = fig.add_subplot(3, 3, 4)
ax_4 = fig.add_subplot(3, 3, 6)
ax_5 = fig.add_subplot(3, 4, 10)
ax_6 = fig.add_subplot(5, 5, 25)

ax_1.set(title = 'ax_1', xticks=[], yticks=[])
ax_2.set(title = 'ax_2', xticks=[], yticks=[])
ax_3.set(title = 'ax_3', xticks=[], yticks=[])
ax_4.set(title = 'ax_4', xticks=[], yticks=[])
ax_5.set(title = 'ax_5', xticks=[], yticks=[])
ax_6.set(title = 'ax_6', xticks=[], yticks=[])

plt.show()
```



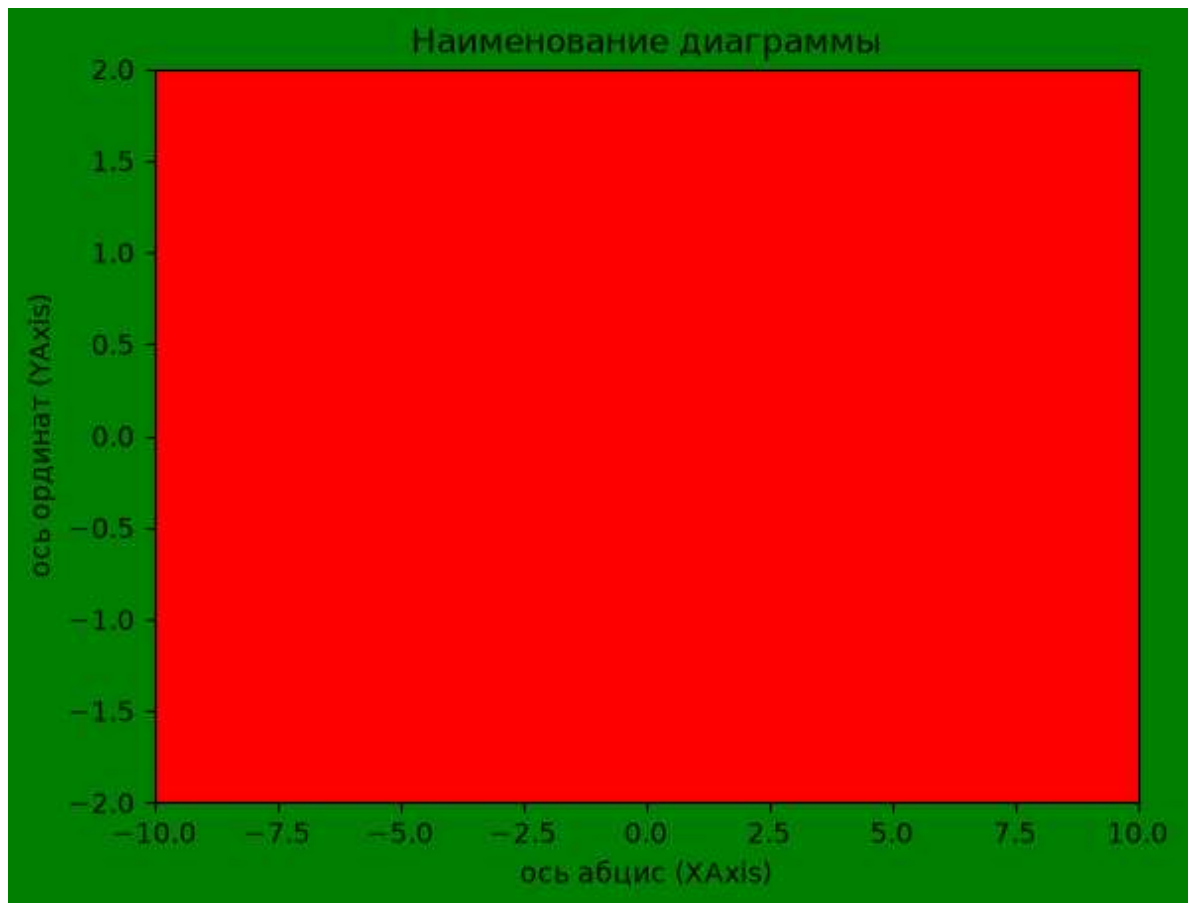
Изменим цвет созданных объектов `Figure` и `Axes` и добавим наименования диаграммы и осей. Для этого используем доступные методы, которые можно выбрать путём нажатия клавиши `TAB` после наименования объекта и точки (`ax.`).

```
In [6]: import matplotlib.pyplot as plt

fig = plt.figure()
ax = fig.add_subplot(111)

fig.set(facecolor = 'green')
ax.set(facecolor = 'red')
ax.set_xlim([-10, 10])
ax.set_ylim([-2, 2])
ax.set_title('Наименование диаграммы')
ax.set_xlabel('ось абцис (XAxis)')
ax.set_ylabel('ось ординат (YAxis)')

plt.show()
```

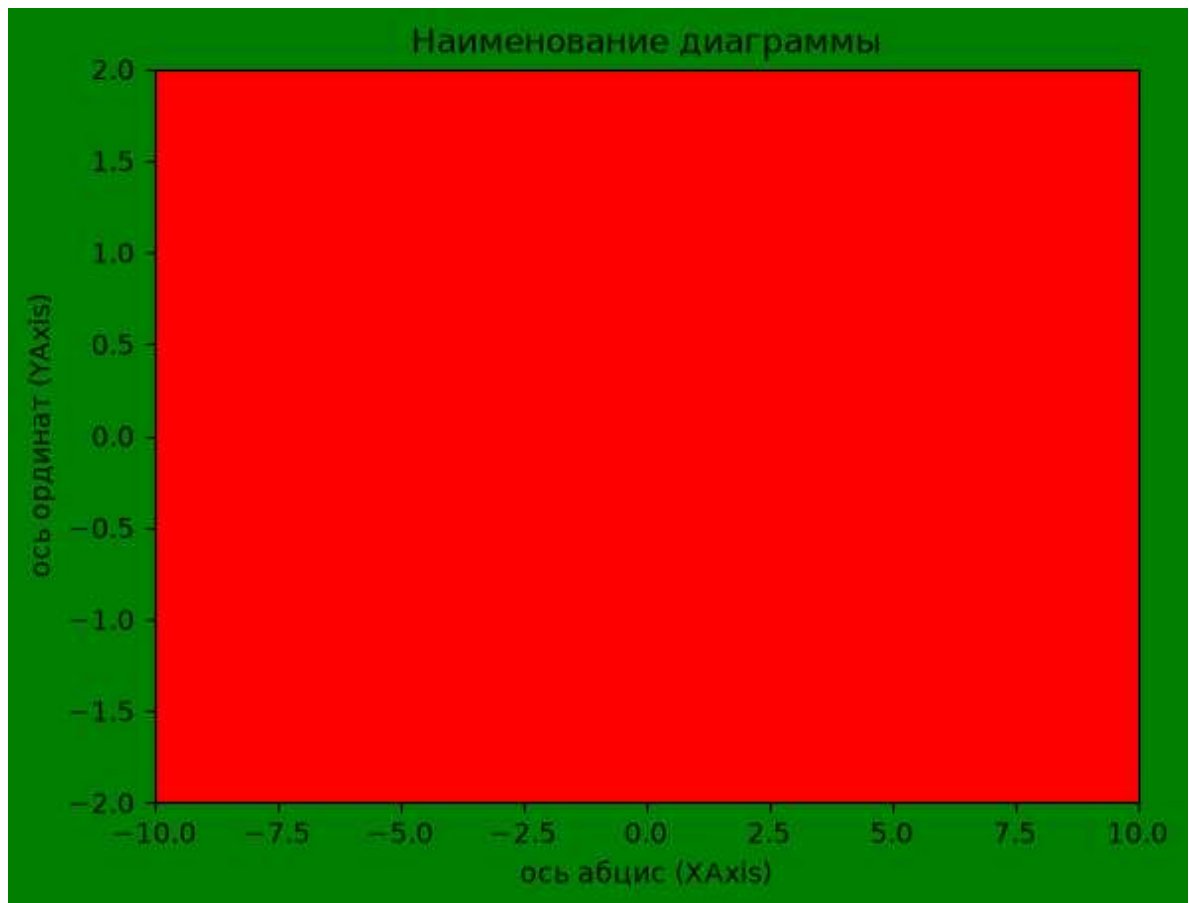


Более короткий способ

```
In [7]: fig = plt.figure()
ax = fig.add_subplot(111)

fig.set(facecolor = 'green')
ax.set(facecolor = 'red',
      xlim = [-10, 10],
      ylim = [-2, 2],
      title = 'Наименование диаграммы',
      xlabel = 'ось абцисс (XAxis)',
      ylabel = 'ось ординат (YAxis)')

plt.show()
```



Метод `set` есть практически у всех объектов `matplotlib`. Изменением цвет и размер наименования диаграммы (`title`)

```
In [8]: # Чисел №1:
fig = plt.figure()
ax = fig.add_subplot(111)

fig.set(facecolor = 'green')
ax.set(facecolor = 'red')
ax.set_xlim([-10, 10])
ax.set_ylim([-2, 2])
ax.set_title('Наименование диаграммы')
ax.set_xlabel('ось абцисс (XAxis)')
ax.set_ylabel('ось ординат (YAxis)')
ax.set_title('Наименование диаграммы', color = 'white', size = 20)
plt.show()
```



```
In [9]: # Числ 2:
fig = plt.figure()
ax = fig.add_subplot(111)

fig.set(facecolor = 'green')
ax.set(facecolor = 'red',
      xlim = [-10, 10],
      ylim = [-2, 2],
      title = 'Наименование диаграммы',
      xlabel = 'ось абцис (XAxis)',
      ylabel = 'ось ординат (YAxis)')
ax.set_title('Наименование диаграммы')
ax.title.set_color('white')
ax.title.set_size(20)
plt.show()
```

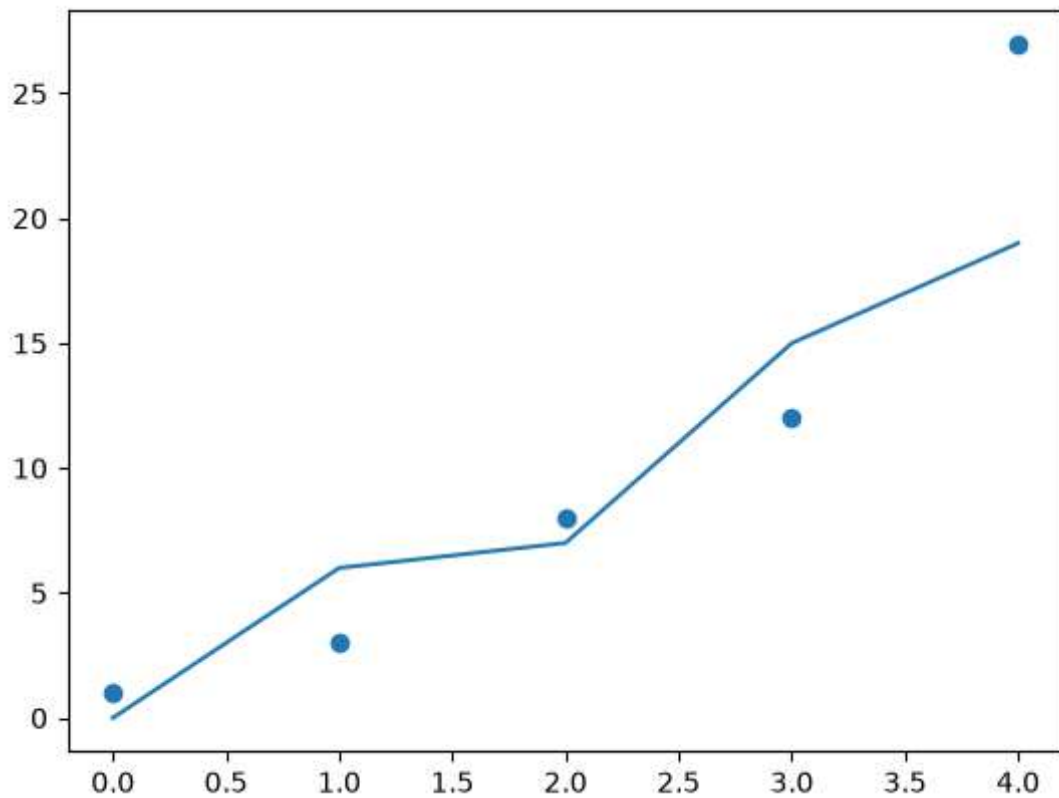


2. Отображение данных на графике

In [10]: `import matplotlib.pyplot as plt`

```
fig = plt.figure()
ax = fig.add_subplot(111)
ax.plot([0, 1, 2, 3, 4], [0, 6, 7, 15, 19]) #линейная диаграмма
ax.scatter([0, 1, 2, 3, 4], [1, 3, 8, 12, 27]) #точечная диаграмма

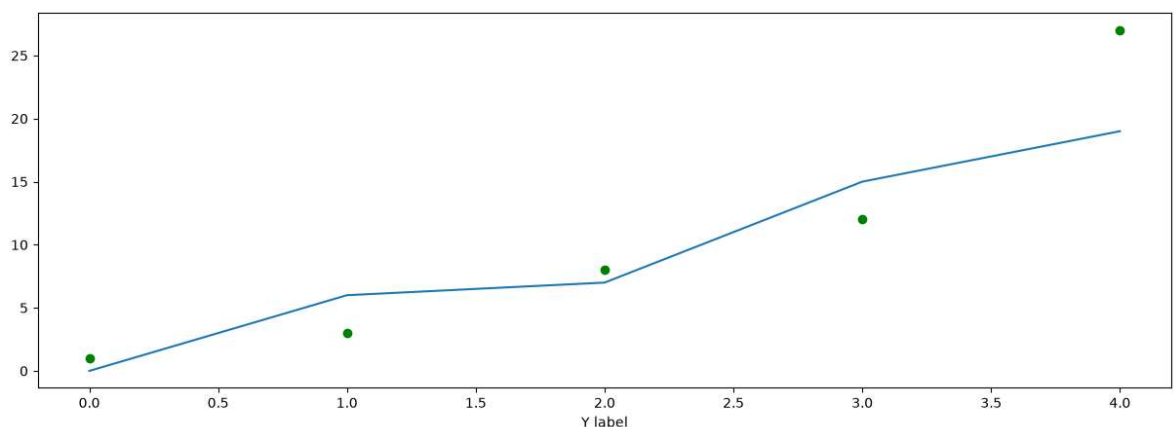
plt.show()
```



Всё остальное можно сделать и другими способами. Важно понять один из них и использовать в практике.

In [11]: `import matplotlib.pyplot as plt`

```
plt.figure(figsize = (15,5)) #размер по горизонтали и вертикали
plt.plot([0, 1, 2, 3, 4], [0, 6, 7, 15, 19])
plt.plot([0, 1, 2, 3, 4], [1, 3, 8, 12, 27], "go")
plt.xlabel("X label")
plt.ylabel("Y label")
plt.show()
```



In [12]: `import matplotlib.pyplot as plt`

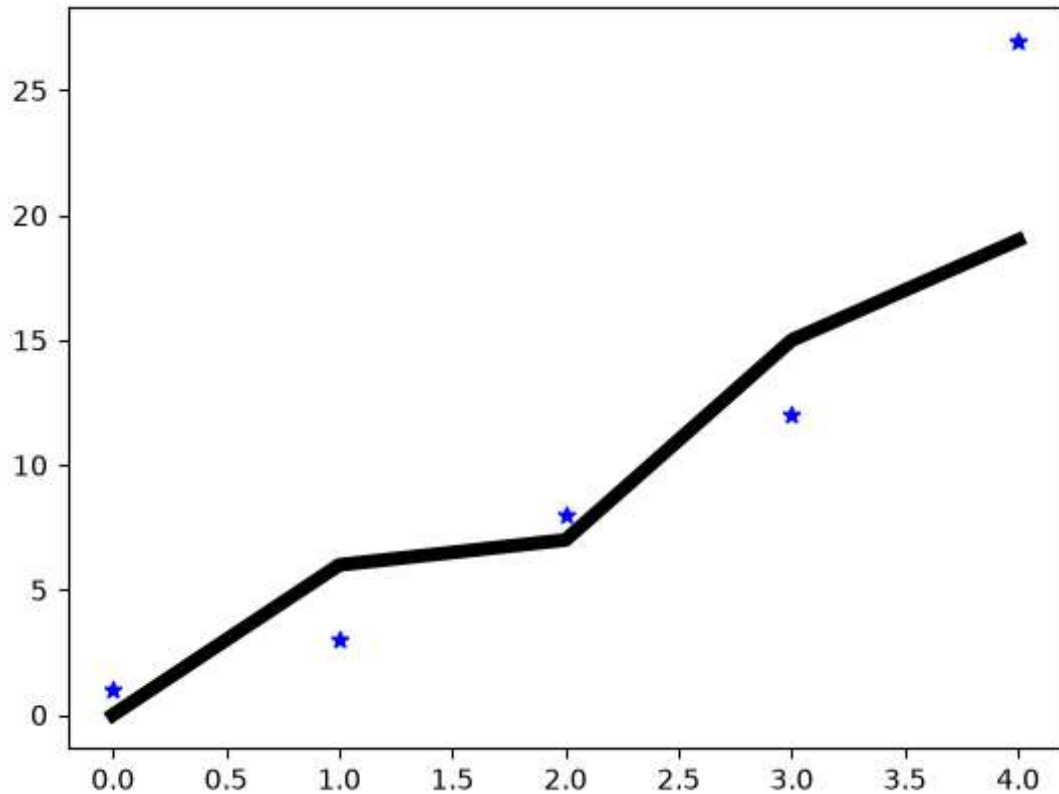
```
x = [0, 1, 2, 3, 4]
y_1 = [0, 6, 7, 15, 19]
y_2 = [1, 3, 8, 12, 27]

fig = plt.figure()
ax = fig.add_subplot(111)
```

```
ax.plot(x, y_1,
        color = 'black',
        linewidth = 5)

ax.scatter(x, y_2,
           color = 'blue',
           marker = '*')

plt.show()
```

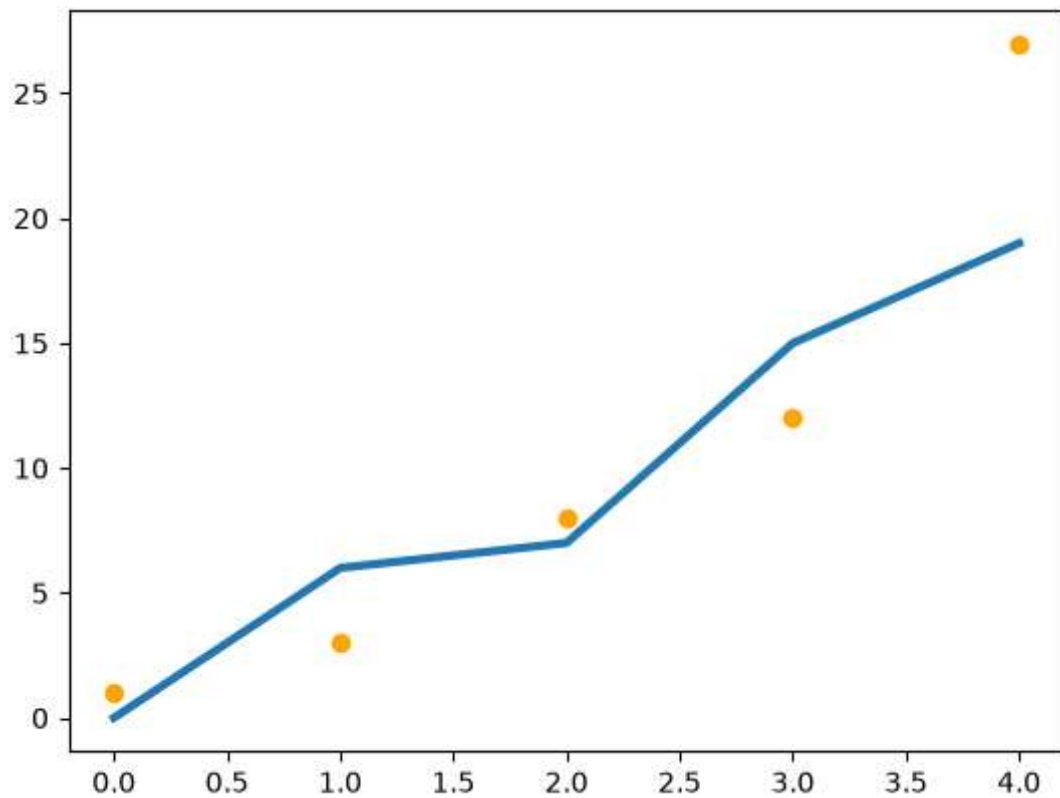


При построении графиков может не требоваться предварительное создание `Figure` и `Axes`. Вызываемые методы могут вызывать методы создания `Figure` и `Axes` самостоятельно.

```
In [13]: import matplotlib.pyplot as plt

plt.plot([0, 1, 2, 3, 4], [0, 6, 7, 15, 19], linewidth = 3)
plt.scatter([0, 1, 2, 3, 4], [1, 3, 8, 12, 27], color = 'orange')

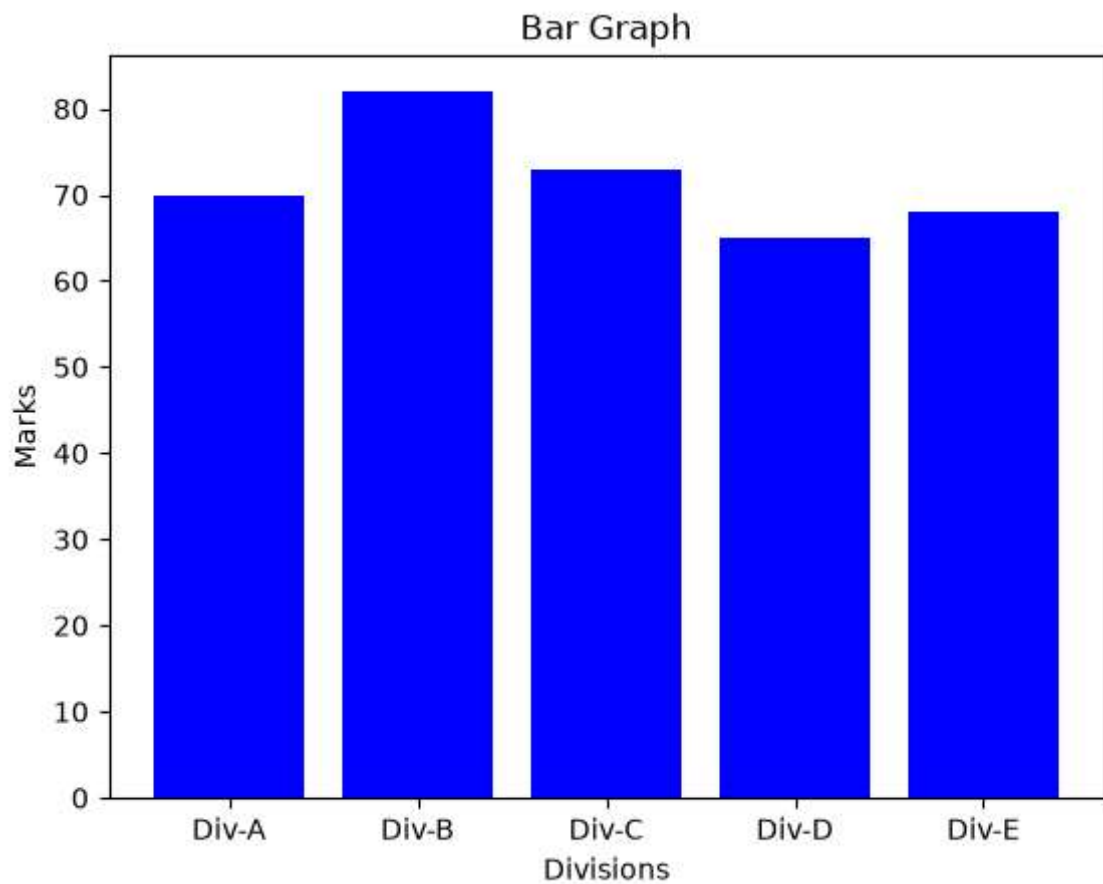
plt.show()
```



3. Разные типы диаграмм

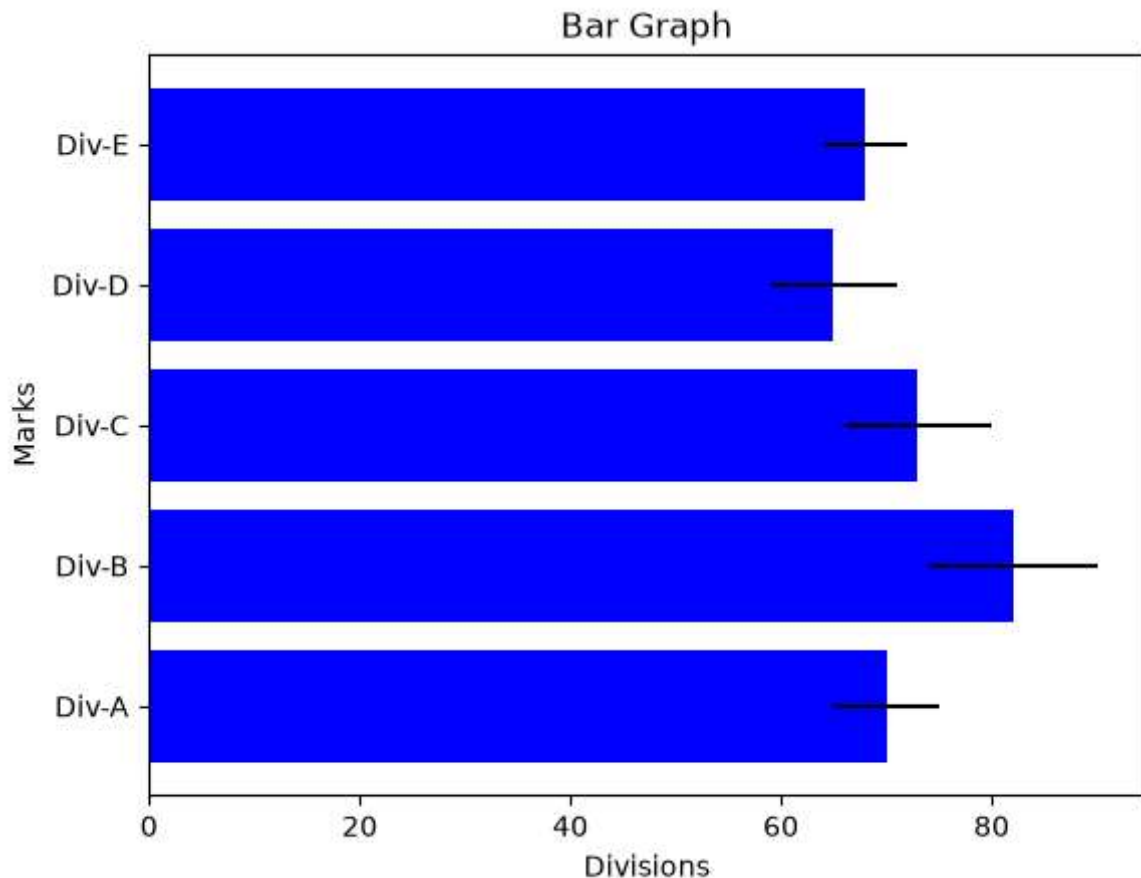
Вертикальная столбчатая гистограмма `bar`

```
In [14]: import matplotlib.pyplot as plt
divisions = ["Div-A", "Div-B", "Div-C", "Div-D", "Div-E"]
division_average_marks = [70, 82, 73, 65, 68]
plt.bar(divisions, division_average_marks, color = 'blue')
plt.title("Bar Graph")
plt.xlabel("Divisions")
plt.ylabel("Marks")
plt.show()
```



Горизонтальная столбчатая гистограмма с бари `barh`

```
In [15]: import matplotlib.pyplot as plt
divisions = ["Div-A", "Div-B", "Div-C", "Div-D", "Div-E"]
division_average_marks = [70, 82, 73, 65, 68]
variance = [5, 8, 7, 6, 4]
plt.barh(divisions, division_average_marks, xerr=variance, color = 'blue')
plt.title("Bar Graph")
plt.xlabel("Divisions")
plt.ylabel("Marks")
plt.show()
```



Два параметра на одной гистограмме

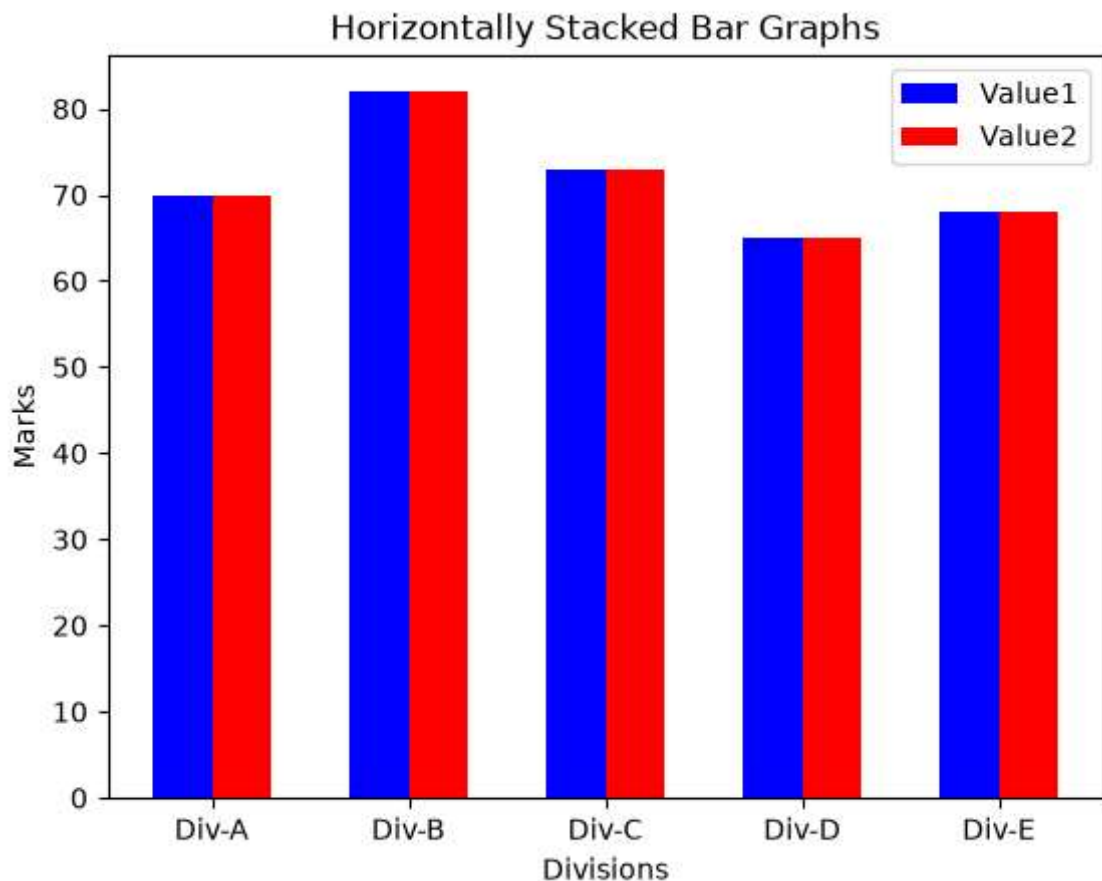
```
In [16]: import matplotlib.pyplot as plt
import numpy as np

divisions = ["Div-A", "Div-B", "Div-C", "Div-D", "Div-E"]
division_average_marks = [70, 82, 73, 65, 68]
boys_average_marks = [68, 67, 77, 61, 70]

index = np.arange(5)
width = 0.30

plt.bar(index, division_average_marks, width, color = 'blue', label = "Value1")
plt.bar(index+width, division_average_marks, width, color = 'red', label = "Value2")
plt.title("Horizontally Stacked Bar Graphs")

plt.xlabel("Divisions")
plt.ylabel("Marks")
plt.xticks(index+width/2, divisions)
plt.legend(loc = 'best')
plt.show()
```



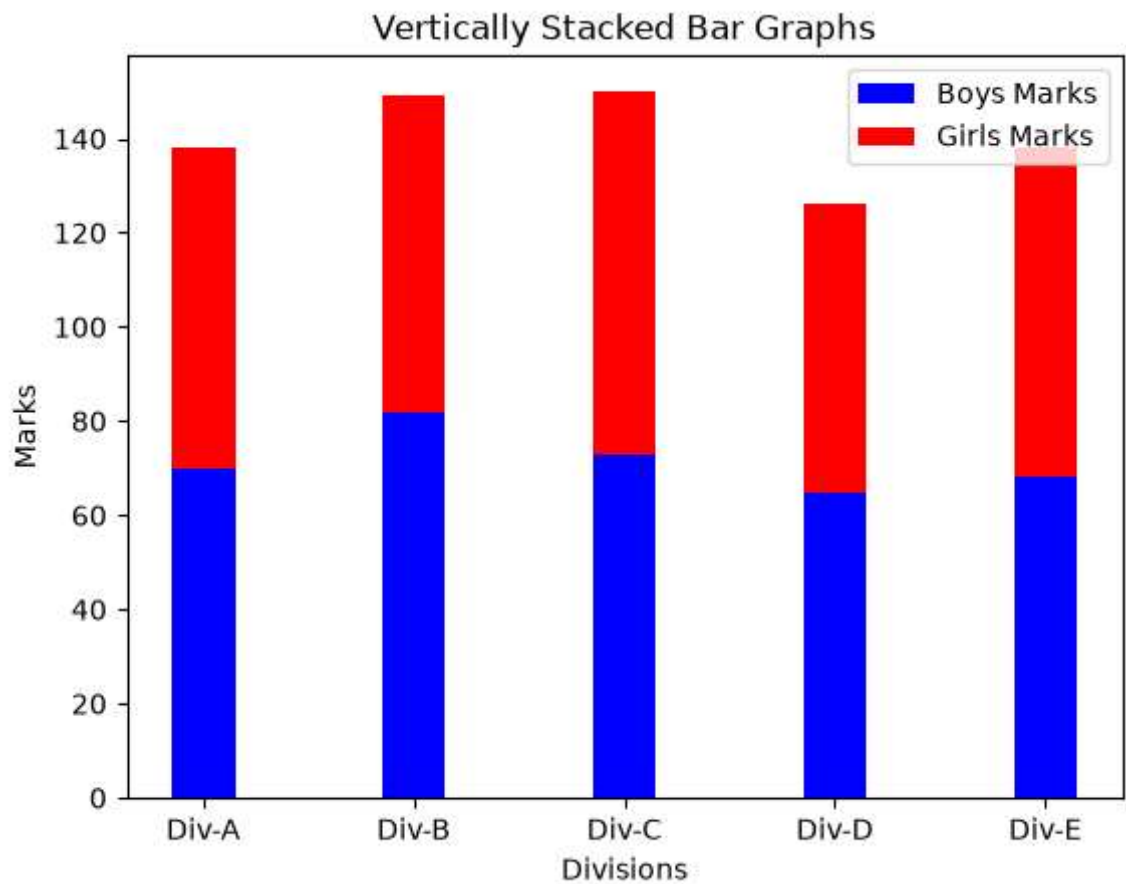
Горизонтальная столбчатая гистограмма с наложением

```
In [17]: divisions = ["Div-A", "Div-B", "Div-C", "Div-D", "Div-E"]
boys_average_marks = [70, 82, 73, 65, 68]
girls_average_marks = [68, 67, 77, 61, 70]

index = np.arange(5)
width = 0.30

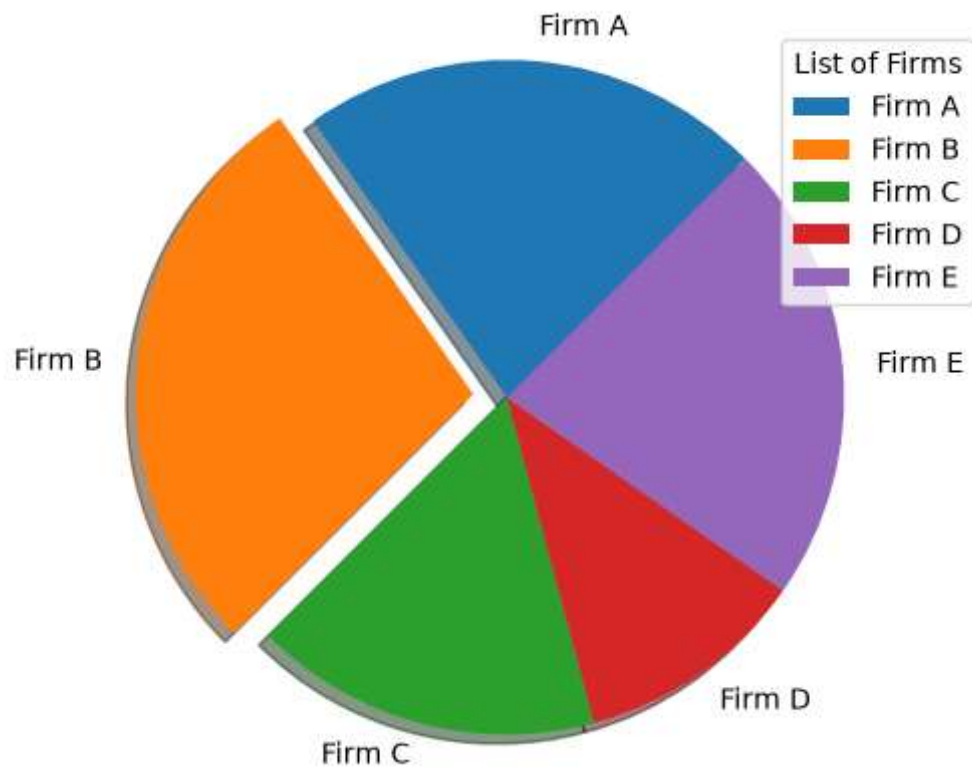
plt.bar(index, division_average_marks, width, color = 'blue', label = "Boys Mar
plt.bar(index, girls_average_marks, width, color = 'red', label = "Girls Marks",
plt.title("Vertically Stacked Bar Graphs")

plt.xlabel("Divisions")
plt.ylabel("Marks")
plt.xticks(index, divisions)
plt.legend(loc = 'best')
plt.show()
```



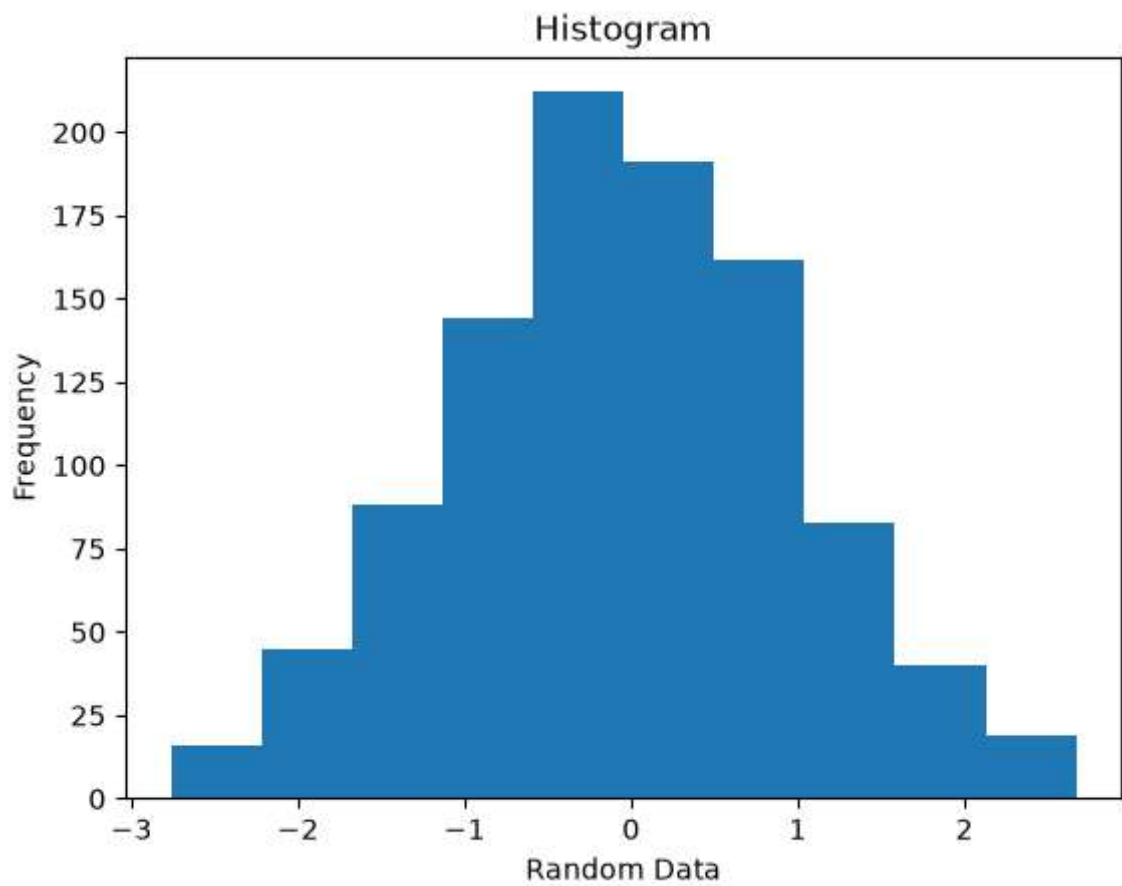
Круговая диаграмма

```
In [18]: firms = ["Firm A", "Firm B", "Firm C", "Firm D", "Firm E"]
market_share = [20, 25, 15, 10, 20]
explode = [0, 0.1, 0, 0, 0]
plt.pie(market_share, explode=explode, labels=firms, shadow=True, startangle=45)
plt.axis('equal')
plt.legend(title="List of Firms")
plt.show()
```



Гистограмма

```
In [19]: import numpy as np
x = np.random.randn(1000)
plt.title("Histogram")
plt.xlabel("Random Data")
plt.ylabel("Frequency")
plt.hist(x,10)
plt.show()
```



Точечный график

```
In [20]: x = np.random.randn(500)
y = np.random.randn(500)
plt.scatter(x, y)
plt.title("Scatter Plot")
plt.xlabel("Height")
plt.xlabel("Weight")
plt.show()
np.random.randn?
```

Scatter Plot

